

CodeNarrator: AI-Driven Solution for Dynamic README Creation and Maintenance

Project Team

Fareeah Naseem 21I-0500
Fatima Khan 21I-0725
Malaika Hussain 21I-2559

Session 2021-2025

Supervised by

Dr. Arshad Islam

Co-Supervised by

Mr. Atif Jilani



Department of Computer Science

**National University of Computer and Emerging Sciences
Islamabad, Pakistan**

June, 2025

Contents

1	Introduction	1
1.1	Problem Statement	1
1.2	Scope	1
1.3	Modules	2
1.3.1	Module 1: Data Collection and Filtering	3
1.3.2	Module 2: Data Preprocessing	4
1.3.3	Module 3: Analysis (Comment and README Analysis)	4
1.3.4	Module 4: Fine-Tuning the NLP Model (CodeBERT)	4
1.3.5	Module 5: Code and README Dissection	4
1.3.6	Module 6: NLP Model Output Generation	5
1.3.7	Module 7: Dynamic Updation	5
1.3.8	Module 8: UI Development	5
1.3.9	Module 9: Integration and Deployment	5
1.4	User Classes and Characteristics	6
2	Project Requirements	7
2.1	Use Cases	7
2.1.1	Use Case 1: Generate Code Comments	8
2.1.2	Use Case 2: View Project Documentation	8
2.1.3	Use Case 3: Generate README	9
2.1.4	Use Case 4: Track Changes and Update Documentation	10
2.1.5	Use Case 5: Integrate with GitHub	11
2.2	Functional Requirements	12
2.2.1	Module 1: Data Collection and Filtering	12
2.2.2	Module 2: Data Preprocessing	12
2.2.3	Module 3: Analysis (Comment and README Analysis)	12
2.2.4	Module 4: Fine-Tuning the NLP Model (CodeBERT)	12
2.2.5	Module 5: Code and README Dissection	13
2.2.6	Module 6: NLP Model Output Generation	13
2.2.7	Module 7: Dynamic Updation	13
2.2.8	Module 8: UI Development	13
2.2.9	Module 9: Integration and Deployment	13

2.3	Non-Functional Requirements	14
2.3.1	Performance	14
2.3.2	Usability	14
2.3.3	Reliability	14
2.3.4	Scalability	15
2.3.5	Security	15
2.3.6	Compatibility	15
2.4	Domain Model	15
2.4.0.1	Relationships between Tables	16
3	System Overview	17
3.1	Architectural Design	17
3.2	Design Methodology	18
3.3	Design Models	19
3.3.1	Activity Diagram	19
3.3.2	Class-level Sequence Diagram	20
3.3.2.1	Generate README	20
3.3.2.2	Generate Code Comments	20
3.3.2.3	Track Changes and Update Documentation	21
3.3.2.4	View Project Documentation	21
3.3.2.5	Integrate with GitHub	21
3.3.3	Component Diagram	22
3.3.4	Flow Diagram	23
3.3.5	Layer Diagram	24
3.3.6	Structure Chart	25
3.3.7	Data Representation Diagram	25
3.4	Data Design	26
3.4.1	Data Dictionary	26
3.4.1.1	Developer Table	26
3.4.1.2	Repository Table	27
3.4.1.3	Commit Table	27
3.4.1.4	README Table	28
3.4.1.5	CodeComment Table	28
3.4.1.6	GitHub Integration Table	29
3.4.1.7	Relationships between Tables	29
	References	31

List of Figures

1.1	Workflow Diagram of the Project	3
2.1	Use Case Diagram for the system	7
2.2	Class Diagram for CodeNarrator	16
3.1	Architectural Diagram of CodeNarrator showing the major subsystems and their interactions across the UI, Application, and Data Layers.	18
3.2	Activity Diagram for CodeNarrator	19
3.3	Class-level Sequence Diagram for Generating README	20
3.4	Class-level Sequence Diagram for Generating Code Comments	20
3.5	Class-level Sequence Diagram for Tracking Changes and Updating Doc- umentation	21
3.6	Class-level Sequence Diagram for Viewing Project Documentation	21
3.7	Class-level Sequence Diagram for Integrating with GitHub	22
3.8	Component Diagram for CodeNarrator showing the JSON schema struc- ture of the repository data.	22
3.9	Flow Diagram for CodeNarrator	23
3.10	Layer Diagram for CodeNarrator	24
3.11	Structure Chart for CodeNarrator	25
3.12	Data Representation Diagram for CodeNarrator	26

List of Tables

- 1.1 User Classes and Their Needs 6
- 3.1 Developer Table 26
- 3.2 Repository Table 27
- 3.3 Commit Table 27
- 3.4 README Table 28
- 3.5 CodeComment Table 28
- 3.6 GitHub Integration Table 29

Chapter 1

Introduction

This project proposal describes the history, difficulties, inspiration, and creative solution that went into creating CodeNarrator. CodeNarrator is an AI-powered tool that streamlines the process of creating and maintaining README files. It is intended to help GitHub projects with their documentation by automating these tasks.test.

1.1 Problem Statement

Due to time and resource restrictions, software developers frequently neglect to maintain complete and up-to-date documentation, particularly README files. Upon initially accessing a repository, developers and contributors generally depend on the README file to gain a high-level understanding of the project. Understanding the goal, organization, and use of the project becomes challenging in the absence of precise and comprehensive documentation. In order to contribute to or incorporate the code into their own work, developers must also delve into the code itself, which is where having thorough code comments becomes crucial. But often, there are insufficient or missing comments, which makes it even harder to understand the code. In order to overcome these difficulties, CodeNarrator offers an automated solution that creates code comments to improve code readability in addition to dynamically creating and updating README files. This guarantees that the code-level documentation and project overview will always be current and available to developers and contributors.

1.2 Scope

This project aims to automate the development and maintenance of README files and code comments for software projects. CodeNarrator ensures that documentation is thor-

ough, up to date, and aligned with the growing codebase by analyzing code, libraries, comments and commit histories. The system will:

- **Generate README files** automatically based on code structure, libraries and dependencies.
- **Dynamically update README files** to ensure correctness based on changes in codebase and commit history.
- **Automatically add pertinent code comments** to the source code to help developers understand its operation and structure.
- **Provide seamless connectivity with GitHub**, allowing for real-time updates and an intuitive interface for developers to manage documentation and code comments.

This project attempts to eliminate manual documentation maintenance and ensure consistency with code changes, both at the high-level (README) and low-level (code comments).

1.3 Modules

The suggested project is broken up into multiple modules, each of which focuses on a distinct facet of the solution. The following are the modules:

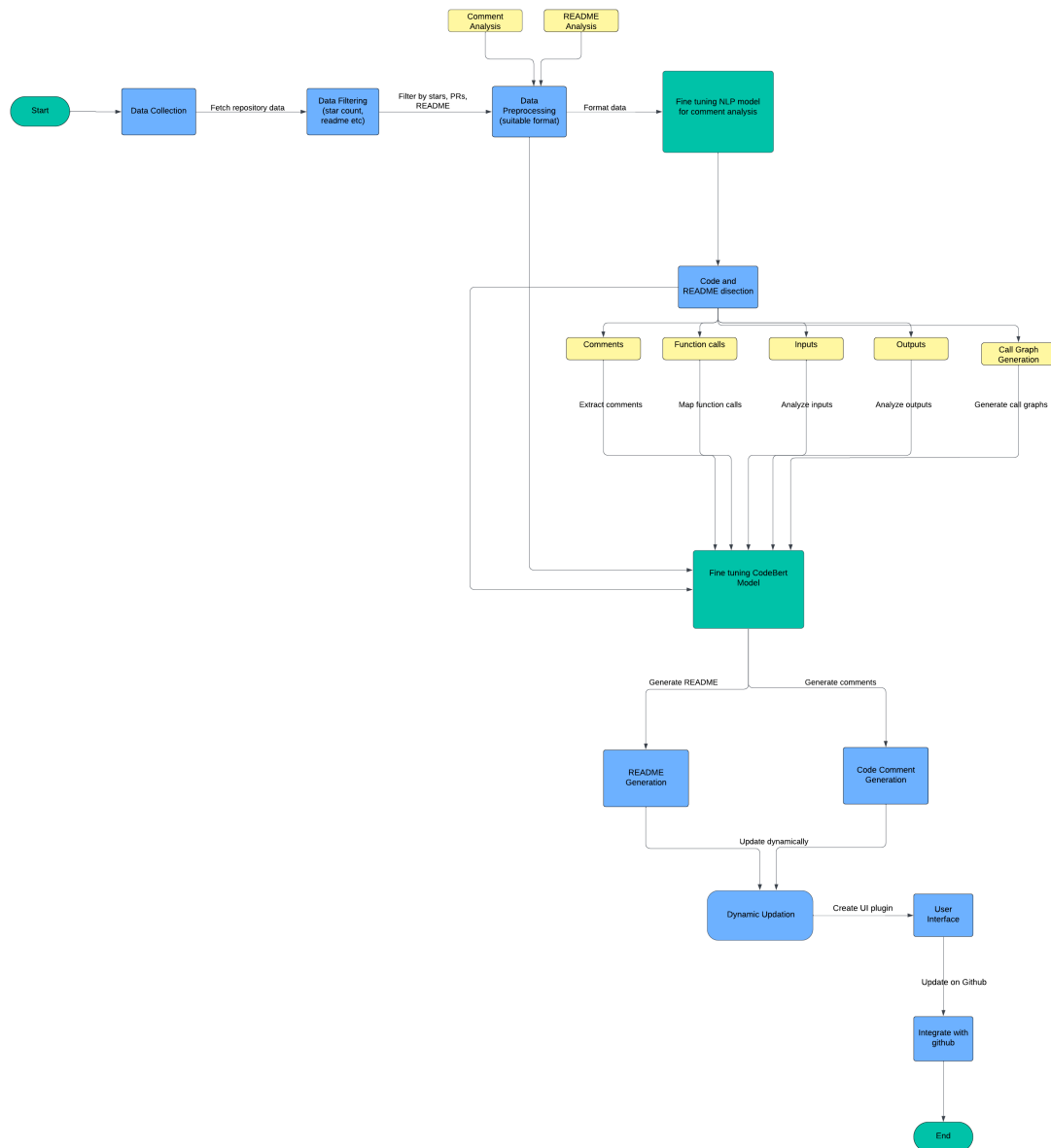


Figure 1.1: Workflow Diagram of the Project

1.3.1 Module 1: Data Collection and Filtering

- **Sub-module 1.1: Retrieve Repository Information**

- Feature: A method to obtain data (from GitHub) by connecting to the reposi-

tory via an interface.

- **Sub-module 1.2: Data Filtering**

- Feature 1: Use the file size to filter data.
- Feature 2: Filter data according to the repository's pull requests, README, or commit frequency.

1.3.2 Module 2: Data Preprocessing

- **Sub-module 2.1: Data Formatting**

- Feature: Utilize a suitable format (such as CSV, JSON, etc.) to organize the obtained data.

1.3.3 Module 3: Analysis (Comment and README Analysis)

- **Sub-module 3.1: Comment Analysis**

- Feature 1: Extract the comments from the code in the repository.
- Feature 2: Preprocess comments for additional examination.

- **Sub-module 3.2: README Analysis**

- Feature: README files should be prepped for additional analysis.

1.3.4 Module 4: Fine-Tuning the NLP Model (CodeBERT)

- **Sub-module 4.1: Model Fine-Tuning**

- Feature: To optimize an NLP model (CodeBERT) for this task, use the README data and preprocessed comments.

1.3.5 Module 5: Code and README Dissection

- **Sub-module 5.1: Function Calls Mapping**

- Feature: Map the code-extracted function calls to further use in call graphs.

- **Sub-module 5.2: Input/Output Analysis**

- Feature: Track the code inputs and outputs.

- **Sub-module 5.3: Call Graph Generation**

- Feature: Generate call graphs based on the information deduced in Sub-module 5.1 and 5.2.

1.3.6 Module 6: NLP Model Output Generation

- **Sub-module 6.1: Fine Tuning CodeBERT Model**

- Feature: Fine-tuning the model based on features dissected in code and README.

- **Sub-module 6.2: README Generation**

- Feature: Generate a fresh README file based on the results of the analysis and the model.

- **Sub-module 6.3: Code Comment Generation**

- Feature: Generate meaningful code comments based on the performed analysis.

1.3.7 Module 7: Dynamic Updation

- **Sub-module 7.1: Dynamic README and Comment Updation**

- Feature: Based on the committed changes to the repository, dynamically update the README file.

1.3.8 Module 8: UI Development

- **Sub-module 8.1: UI Plugin Development**

- Feature: Design an interactive plugin for the end users to use the dynamic update and README generating features.

- **Sub-module 8.2: User Interface**

- Feature: Develop a user-friendly and easy-to-use UI for this project.

1.3.9 Module 9: Integration and Deployment

- **Sub-module 9.1: GitHub Integration**

- Feature: Reintegrate the generated results into GitHub (README, comments).

1.4 User Classes and Characteristics

This section identifies the different user classes that will interact with the product, detailing their key characteristics, needs, and how the system addresses their specific requirements

User Class	Description	Needs
Developers	Programmers working on GitHub projects need to frequently update documentation and code comments while managing tight deadlines and complex projects. Their work often involves multiple tools and manual updates.	Automated README and code comments generation to simplify their workflow and integrate with GitHub for seamless, minimal-effort documentation updates.
Development Teams	Teams with varying levels of experience rely on clear, consistent documentation to ensure collaboration and project scalability. Inconsistent documentation affects onboarding and project handover.	Consistent, well-organized documentation that promotes efficient teamwork, reduces knowledge gaps, and ensures uniformity across the project.
Open-Source Contributors	Contributors who are unfamiliar with the internal workings of the project rely heavily on clear README files and comments to understand the project quickly and contribute effectively.	Up-to-date and easy-to-understand documentation that provides a high-level overview of the project and assists with onboarding.
Project Managers	Managers need accurate and up-to-date documentation to monitor progress, assign tasks, and facilitate project handover and scalability. They focus on overall project coordination and resource management.	Comprehensive documentation that includes real-time updates to track progress and reduce the manual effort required to oversee the team's work.
Quality Assurance Teams	QA teams need to validate the quality and integrity of the code by identifying critical areas that require testing. They rely on documentation to understand code changes and ensure completeness in testing.	Documentation and comments that are precise and up-to-date, helping QA teams focus on testing the latest code effectively.

Table 1.1: User Classes and Their Needs

Chapter 2

Project Requirements

This chapter describes the functional and non-functional requirements of the project.

2.1 Use Cases

Below is the Use Case Diagram illustrating the interactions between users and the system:

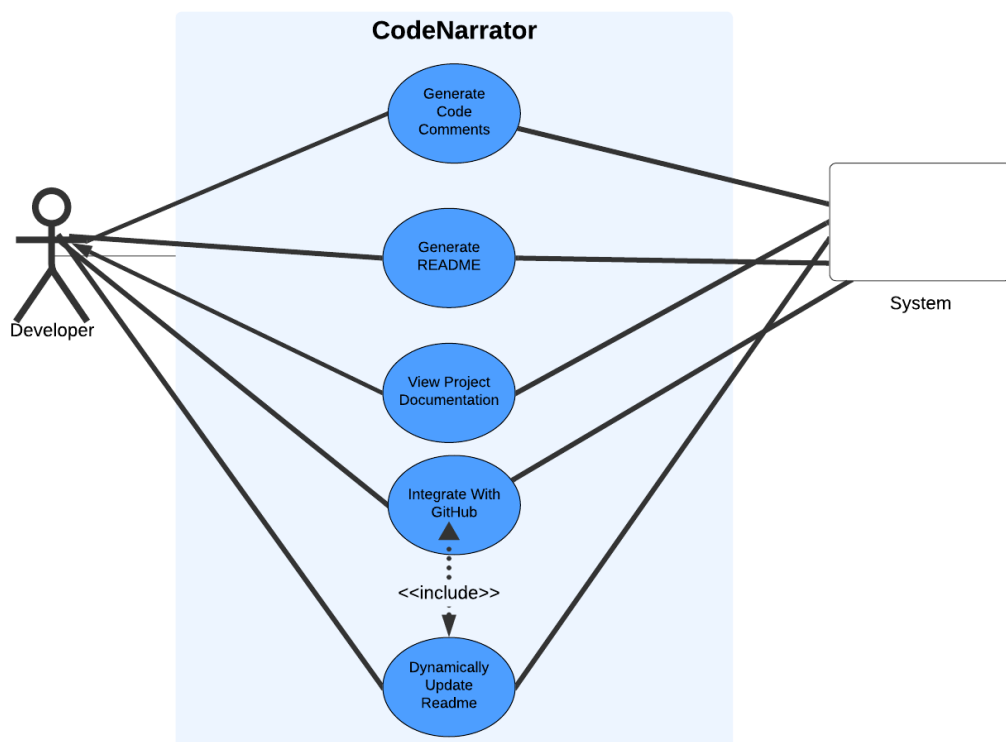


Figure 2.1: Use Case Diagram for the system

2.1.1 Use Case 1: Generate Code Comments

Primary Actor: User

Secondary Actor: CodeNarrator System

Precondition: The user is logged into the CodeNarrator system, and the code is available in the repository.

Postcondition: The code is returned to the user with generated comments.

Main Success Scenario:

1. The **User** selects a piece of code for which they want to generate comments.
2. The **User** sends a request to the **CodeNarrator System** to generate comments.
3. The **CodeNarrator System** fetches the selected code from the repository.
4. The system analyzes the code and generates appropriate comments.
5. The system returns the commented code to the **User**.
6. The **User** views the commented code.

Extensions:

- **1a:** The code does not exist in the repository. The system notifies the user that the code cannot be found.

Special Requirements:

- The system should generate comments following the coding standards specified in the repository settings.
- The process should complete in under 5 seconds for a typical file size of less than 1MB.

2.1.2 Use Case 2: View Project Documentation

Primary Actor: User

Secondary Actor: CodeNarrator System

Precondition: The project README and comments are available in the repository.

Postcondition: The system displays the project documentation to the user.

Main Success Scenario:

1. The **User** requests to view the project documentation.
2. The **CodeNarrator System** fetches the README and any relevant comments from the repository.
3. The system displays the project documentation (README and comments) to the **User**.
4. The **User** views the project documentation.

Extensions:

- **1a:** The README file is missing. The system notifies the **User** that the README file is not available.
- **1b:** The comments are missing. The system notifies the **User** that there are no comments to display.

Special Requirements:

- The system should format the README file and comments properly for easy viewing.
- The response time for displaying the documentation should be under 3 seconds.

2.1.3 Use Case 3: Generate README

Primary Actor: User

Secondary Actor: CodeNarrator System

Precondition: The project repository exists, and data is available for analysis.

Postcondition: The system generates and returns a README file to the user.

Main Success Scenario:

1. The **User** requests the **CodeNarrator System** to generate a README for their project.
2. The system fetches the required data from the repository.
3. The system analyzes the repository structure, code, and commit history.
4. The system generates a comprehensive README file based on the analysis.
5. The system returns the generated README to the **User**.

Extensions:

- **1a:** The system cannot fetch the repository data. The system notifies the **User** that the repository data could not be retrieved.
- **2a:** The system encounters incomplete or incorrect data while generating the README. The system notifies the **User** about potential issues in the generated README.

Special Requirements:

- The README should include information on project description, installation, usage, contributors, and license (if applicable).
- The generation process should not take more than 10 seconds for a typical repository.

2.1.4 Use Case 4: Track Changes and Update Documentation

Primary Actor: GitHub (acts as a notifying entity)

Secondary Actor: CodeNarrator System

Precondition: A new commit is made to the repository.

Postcondition: The system updates the documentation based on the commit and syncs the changes.

Main Success Scenario:

1. **GitHub** notifies the **CodeNarrator System** of a new commit in the repository.
2. The system fetches the changes from the new commit.
3. The system updates the README and comments based on the changes in the commit.
4. The system synchronizes the updated documentation with the repository.
5. The **User** can view the updated documentation.

Extensions:

- **1a:** The system fails to fetch the commit changes. The system notifies the **User** of the failure.
- **2a:** The commit changes do not affect the README or comments. The system does not perform any updates but notifies the **User** that no updates were necessary.

Special Requirements:

- The system should automatically check for new commits every 10 minutes.
- The updated documentation should be synchronized within 2 minutes of detecting the new commit.

2.1.5 Use Case 5: Integrate with GitHub

Primary Actor: User

Secondary Actor: GitHub Marketplace, CodeNarrator System

Precondition: The user has access to GitHub and the GitHub Marketplace.

Postcondition: The CodeNarrator app is installed and integrated with GitHub, and permissions are granted.

Main Success Scenario:

1. The **User** navigates to the **GitHub Marketplace**.
2. The **User** adds the **CodeNarrator app** to their GitHub account.
3. The **GitHub Marketplace** requests the necessary permissions for the app.
4. The **User** reviews and grants the required permissions.
5. The **GitHub Marketplace** notifies the **User** that the app is installed and access is granted.
6. The **User** can now use the **CodeNarrator app** with their GitHub repository.

Extensions:

- **1a:** The **User** cancels the installation process. The system aborts the integration and notifies the **User**.
- **2a:** The **User** denies the permissions. The system aborts the integration and notifies the **User**.
- **3a:** The installation fails. The system notifies the **User** that the installation failed.

Special Requirements:

- The **User** must have a GitHub account and access to the GitHub Marketplace.
- The permissions should include access to repositories, commits, and documentation.

2.2 Functional Requirements

This section lists all of the system's functional requirements in natural language according to each module.

2.2.1 Module 1: Data Collection and Filtering

- **FR1:** The system will gather repository data from GitHub, including code files, commit history, pull requests, and README content.
- **FR2:** The system will apply filters to the repository data based on file size, pull request frequency, commit history, and the presence of a README file.

2.2.2 Module 2: Data Preprocessing

- **FR1:** The system will format the retrieved data into formats like CSV or JSON to facilitate further analysis.
- **FR2:** The system will ensure the formatted data is well-structured and consistent for efficient processing.

2.2.3 Module 3: Analysis (Comment and README Analysis)

- **FR1:** The system will extract all code comments from the repository for analysis.
- **FR2:** The system will preprocess the extracted comments to prepare them for further evaluation.
- **FR3:** The system will extract key sections from README files to prepare them for in-depth analysis.

2.2.4 Module 4: Fine-Tuning the NLP Model (CodeBERT)

- **FR1:** The system will fine-tune the CodeBERT model using README content and preprocessed comments from the repository.
- **FR2:** The system will optimize the NLP model to improve the generation of meaningful comments and README content.

2.2.5 Module 5: Code and README Dissection

- **FR1:** The system will map function calls extracted from the code for use in generating call graphs.
- **FR2:** The system will track inputs and outputs in the code to identify its structure and logic flow.
- **FR3:** The system will generate call graphs based on the extracted function calls, inputs, and outputs.

2.2.6 Module 6: NLP Model Output Generation

- **FR1:** The system will fine-tune the CodeBERT model based on the analysis of the code structure and README content.
- **FR2:** The system will generate a new README file using the results of the analysis and the fine-tuned model.
- **FR3:** The system will generate meaningful code comments that describe the functionality of the code based on the analysis.

2.2.7 Module 7: Dynamic Updation

- **FR1:** The system will dynamically update the README file to reflect changes committed to the repository.
- **FR2:** The system will update code comments dynamically whenever there are significant changes to the repository.

2.2.8 Module 8: UI Development

- **FR1:** The system will feature a user-friendly interface that makes it easy for users to generate and update documentation.

2.2.9 Module 9: Integration and Deployment

- **FR1:** The system shall automatically push updates to GitHub whenever documentation or code comments are updated.

2.3 Non-Functional Requirements

This section outlines the non-functional requirements that are specific, quantitative, and verifiable.

2.3.1 Performance

- **PER-1:** The system must process and generate documentation within 2-3 minutes for repositories with up to 50 files.
- **PER-2:** The system should update README files and code comments in real-time, immediately following revisions submitted to the repository.

2.3.2 Usability

- **USE-1:** The user interface should be easy to navigate and offer clear options for README creation, comment generation and updates.
- **USE-2:** The system shall provide clear error messages when document generation fails or when integration with GitHub displays issues.
- **USE-3:** The system shall allow the users to generate or change the documentation with a single interaction.

2.3.3 Reliability

- **REL-1:** The system must verify that the generated documentation accurately represents the repository's content, without formatting or data problems.
- **REL-2:** The system shall handle unexpected repository structures or malformed data gracefully by providing fallback mechanisms and error handling.
- **REL-3:** The system's Mean Time Between Failures (MTBF) should be at least 30 days.
- **REL-4:** The system shall automatically retry failed operations up to three times before providing a failure notification.

2.3.4 Scalability

- **SCA-1:** The system be able to handle repositories with up to 100 files and complex directory structures without performance reduction.
- **SCA-2:** The system should scale easily across all GitHub repositories and projects, regardless of their structure.

2.3.5 Security

- **SEC-1:** The system must employ secure API calls (e.g., OAuth) to access repositories. All authentication should be encrypted and adhere to GitHub's security.
- **SEC-2:** Any repository or personal data must be handled in compliance with GitHub's security guidelines and best practices.
- **SEC-3:** The system must ensure that all sensitive data (such as API tokens and personal information) is safely saved and never exposed during the procedure.

2.3.6 Compatibility

- **COM-1:** The system must be compatible with a wide range of GitHub repositories, regardless of the programming languages or project structures used.
- **COM-2:** The user interface must work smoothly with the latest versions of modern web browsers, including Google Chrome, Mozilla Firefox, and Safari.

2.4 Domain Model

The following Class Diagram illustrates the domain model of the system, highlighting the key entities, their attributes, and the relationships between various classes.

CodeNarrator: UML Class Diagram

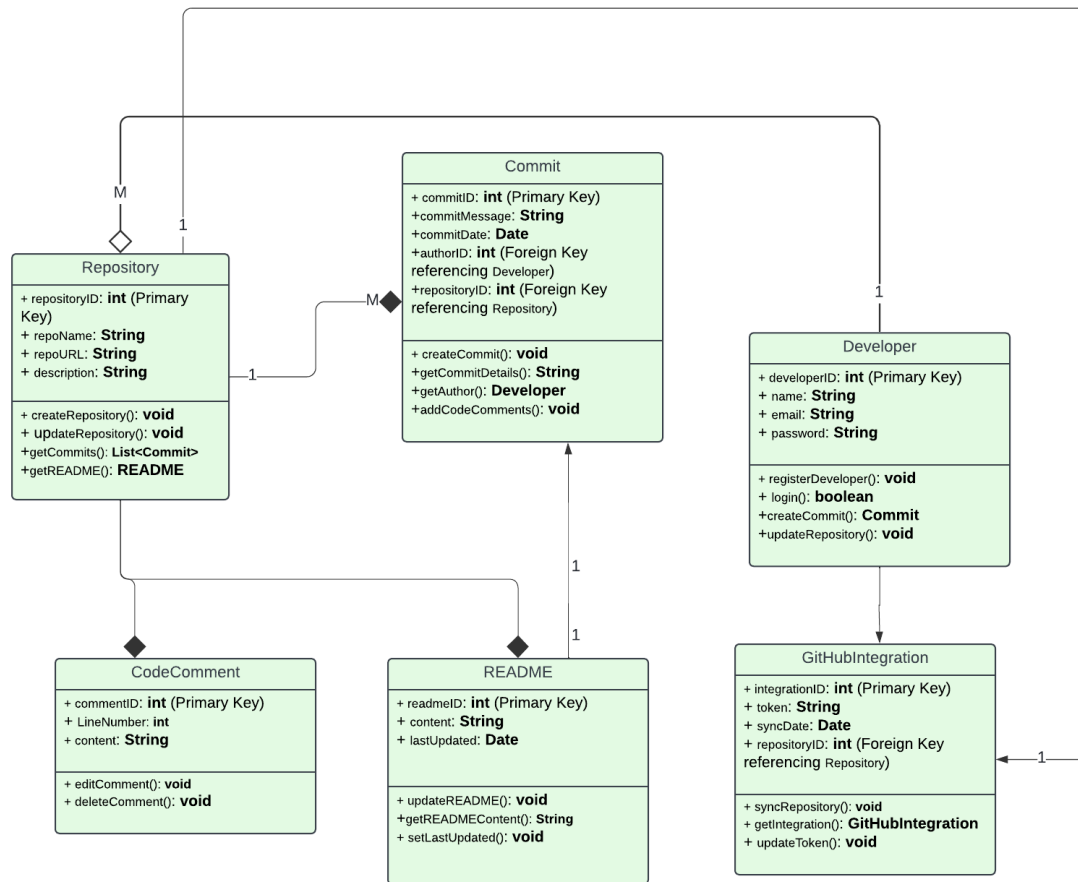


Figure 2.2: Class Diagram for CodeNarrator

2.4.0.1 Relationships between Tables

- **Developer - Repository:** A developer can own multiple repositories (1 : M)
- **Repository - Commit:** A repository can have multiple commits (1 : M)
- **Repository - README:** Each repository has one README file (1 : 1)
- **Repository - CodeComment:** A repository can have multiple code comments (1 : M)
- **Developer - Commit:** A developer can make multiple commits (1 : M)

Chapter 3

System Overview

Give a general description of the functionality, context, and design of your project. Provide any background information if necessary.

3.1 Architectural Design

The architecture of CodeNarrator follows a **layered architecture pattern**, which is well-suited for its modular and scalable nature. This approach provides a clear **separation of concerns**, with each layer responsible for distinct tasks. The system is made up of three layers:

1. **UI Layer:** User interactions are handled by the UI layer. Through the **CodeNarrator Web UI**, users can submit requests to generate README files, update code comments, or update documentation. Serving as the user interface, this layer makes it easier for users to communicate with the application layer.
2. **Application Layer:** Requests and business logic are processed by this layer. The **CodeBERT (NLP)** model is used by the **Request Handler** module to analyze code, handle user commands, and carry out dynamic changes to the README files. Based on commits and changes made to the code in the repository, the **Dynamic Update Handler** makes sure that the comments and documentation are up to date.
3. **Data Layer:** This layer controls communication with outside services and storage of documentation. It synchronizes the generated README files and code comments with the CodeNarrator Database and interfaces with the GitHub API to fetch and push updates to repositories.

This layered approach provides clear responsibility for each component:

- The **UI Layer** handles all user interactions,
- The **Application Layer** processes the logic and commands, and
- The **Data Layer** stores and manages the data.

Additionally, this modularity ensures that each component—such as the GitHub API integration, NLP model (CodeBERT), or dynamic update handler—can be updated independently without affecting the overall system functionality.

Below is the architectural diagram illustrating the system’s modular structure and the interactions between different layers:

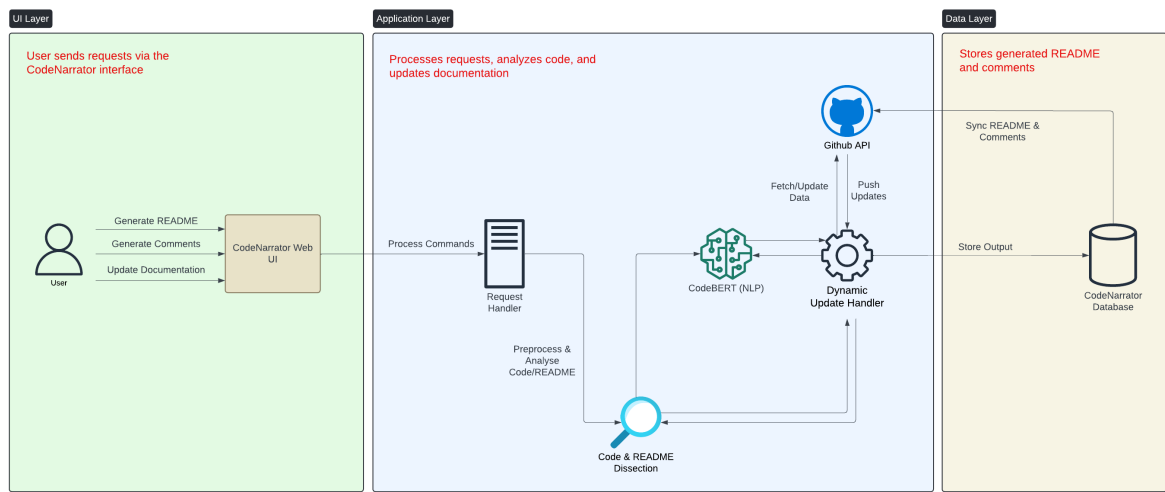


Figure 3.1: Architectural Diagram of CodeNarrator showing the major subsystems and their interactions across the UI, Application, and Data Layers.

3.2 Design Methodology

Code Narrator’s layered architecture aligns perfectly with the **Object-Oriented (OO) methodology**. Each layer—UI, Application, and Data—follows the OO principles of **modularity** and **encapsulation**, making components like the **Request Handler**, **CodeBERT NLP**, and **Dynamic Update Handler** independent and manageable. The clear **component interaction** ensures seamless communication between modules, while the system’s **scalability** allows new features to be added without disrupting existing functionality. This combination of OO and layered design provides flexibility, maintainability, and supports real-time updates efficiently.

3.3 Design Models

The following design models showcase the system's structure, interactions, and data flow, essential for object-oriented development.

3.3.1 Activity Diagram

Below is the Activity Diagram representing the system's workflow:

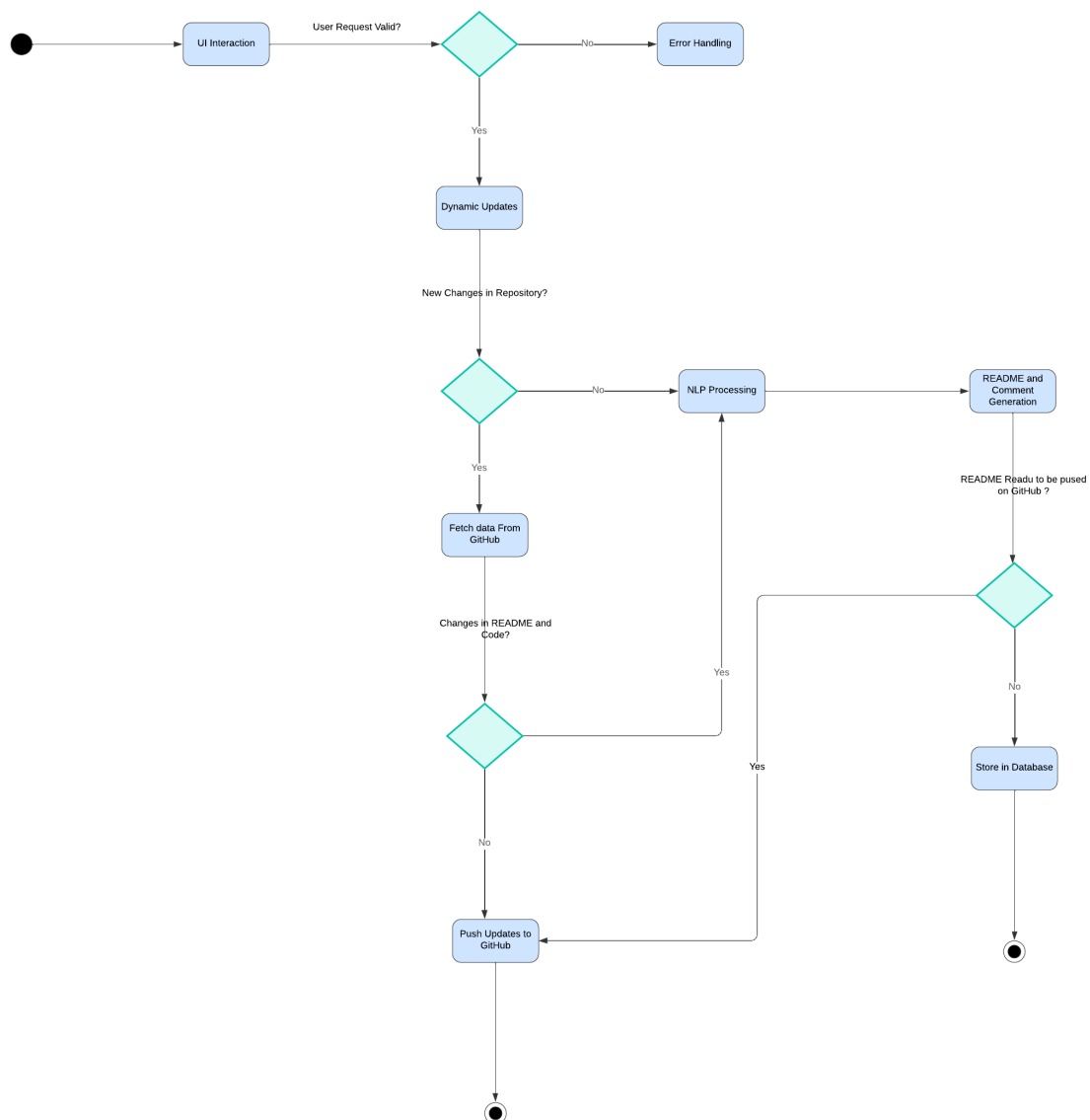


Figure 3.2: Activity Diagram for CodeNarrator

3.3.2 Class-level Sequence Diagram

Below are the Class-level Sequence Diagrams showing the sequence of operations between classes:

3.3.2.1 Generate README

Below is the sequence diagram for generating the README file:

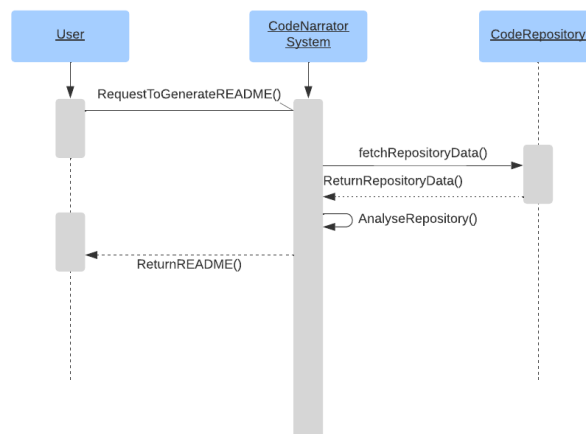


Figure 3.3: Class-level Sequence Diagram for Generating README

3.3.2.2 Generate Code Comments

Below is the sequence diagram for generating code comments:

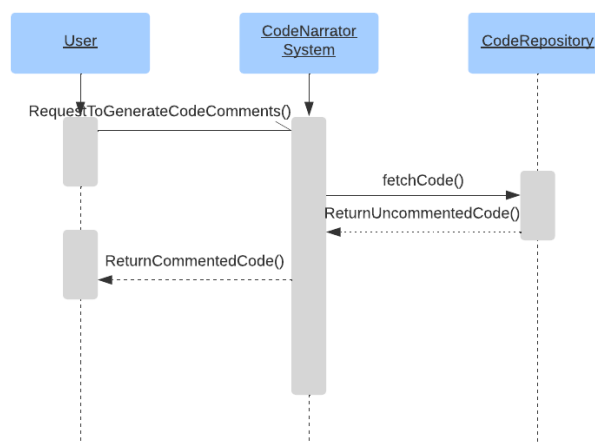


Figure 3.4: Class-level Sequence Diagram for Generating Code Comments

3.3.2.3 Track Changes and Update Documentation

Below is the sequence diagram for tracking changes and updating documentation:

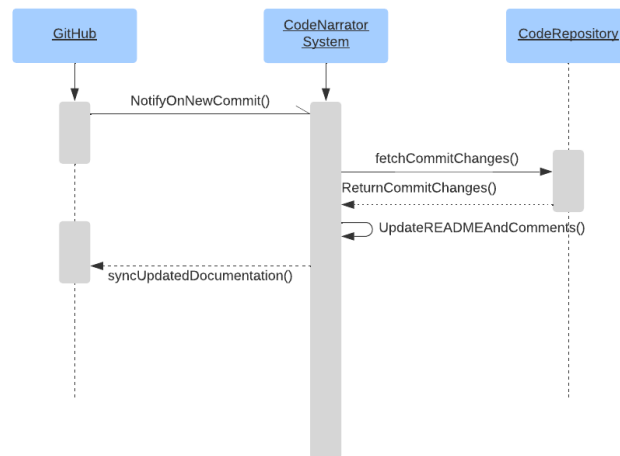


Figure 3.5: Class-level Sequence Diagram for Tracking Changes and Updating Documentation

3.3.2.4 View Project Documentation

Below is the sequence diagram for viewing project documentation:

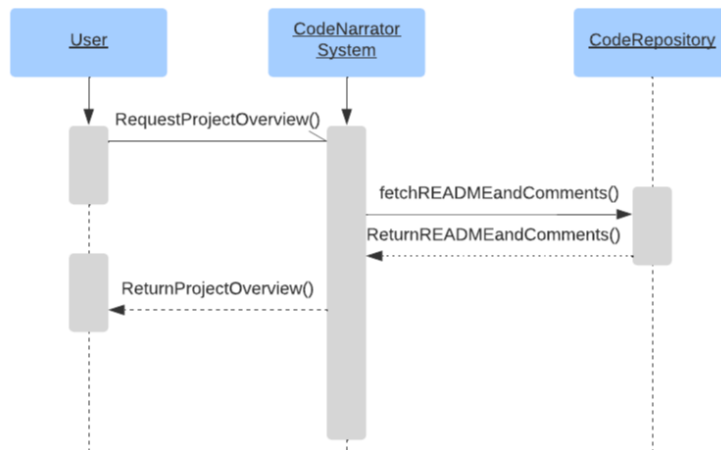


Figure 3.6: Class-level Sequence Diagram for Viewing Project Documentation

3.3.2.5 Integrate with GitHub

Below is the sequence diagram for integrating with GitHub:

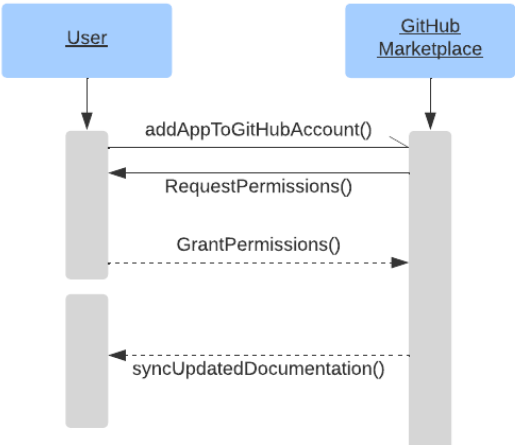


Figure 3.7: Class-level Sequence Diagram for Integrating with GitHub

3.3.3 Component Diagram

Below is the Component Diagram showing the various components of the system:

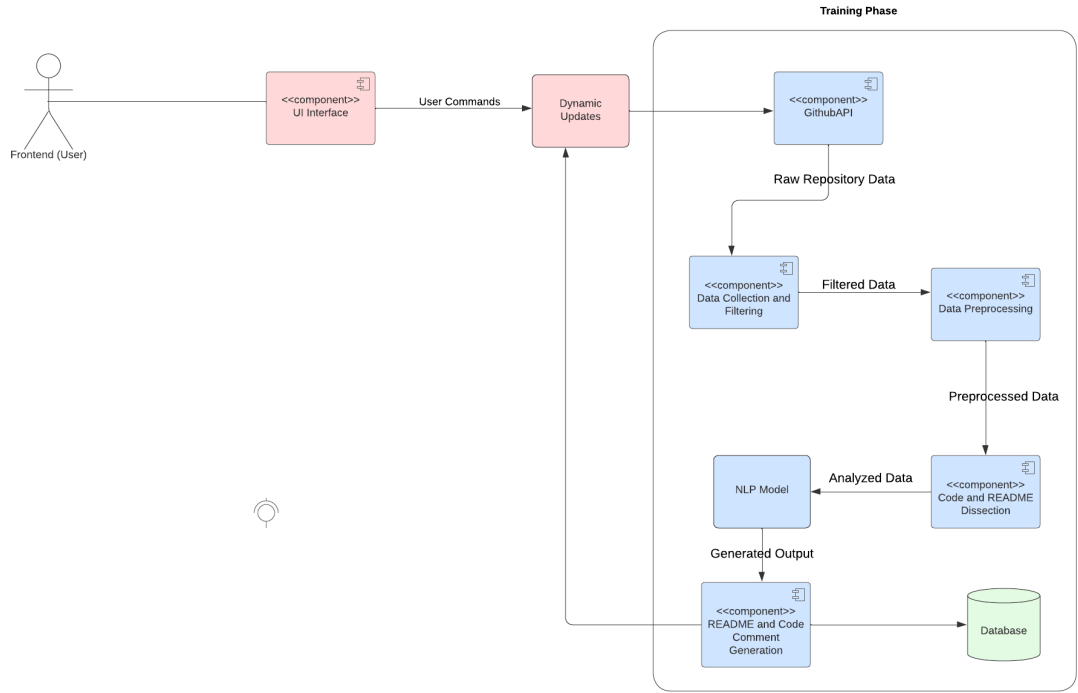


Figure 3.8: Component Diagram for CodeNarrator showing the JSON schema structure of the repository data.

3.3.4 Flow Diagram

Below is the Flow Diagram showing the data flow through the system:

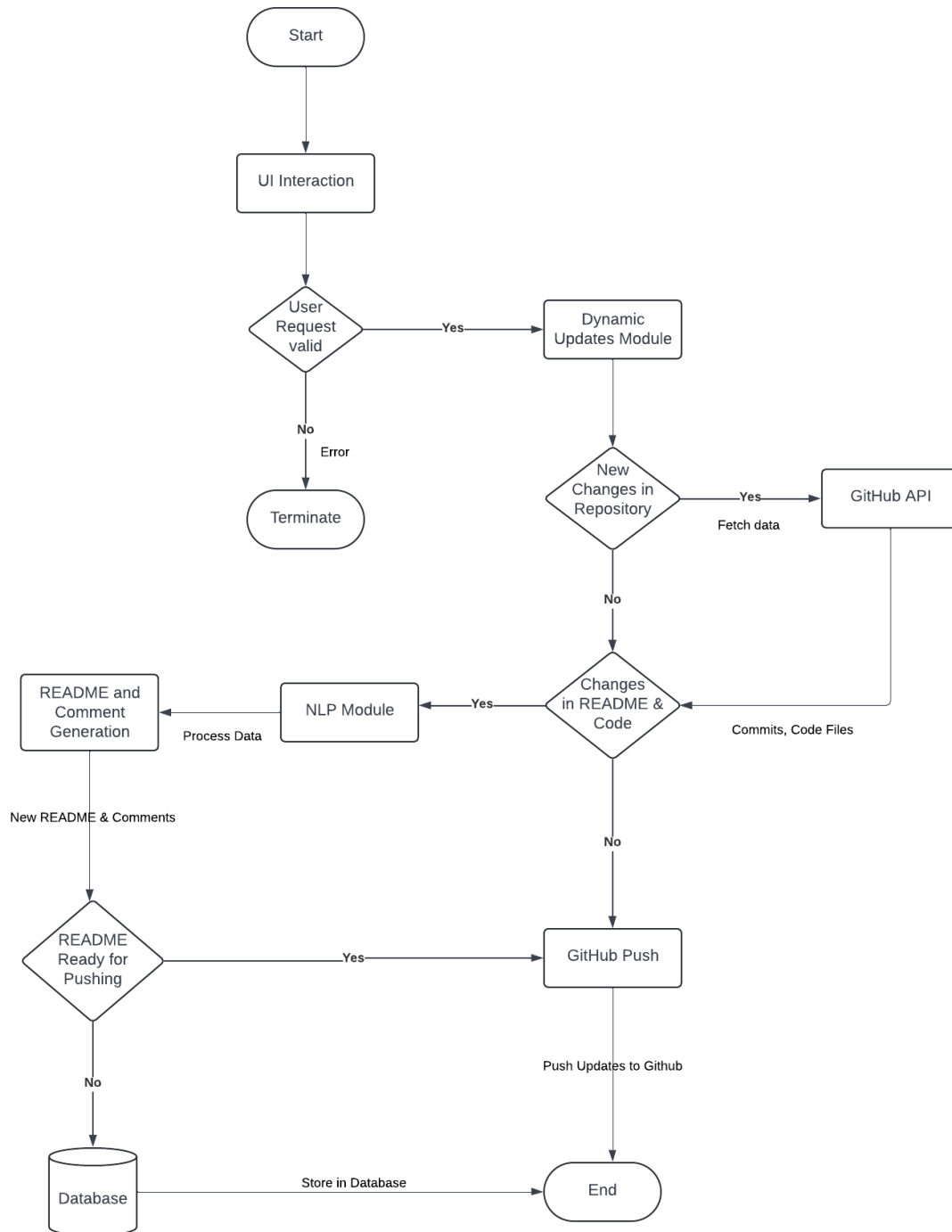


Figure 3.9: Flow Diagram for CodeNarrator

3.3.5 Layer Diagram

Below is the Layer Diagram showing the layered architecture of the system:

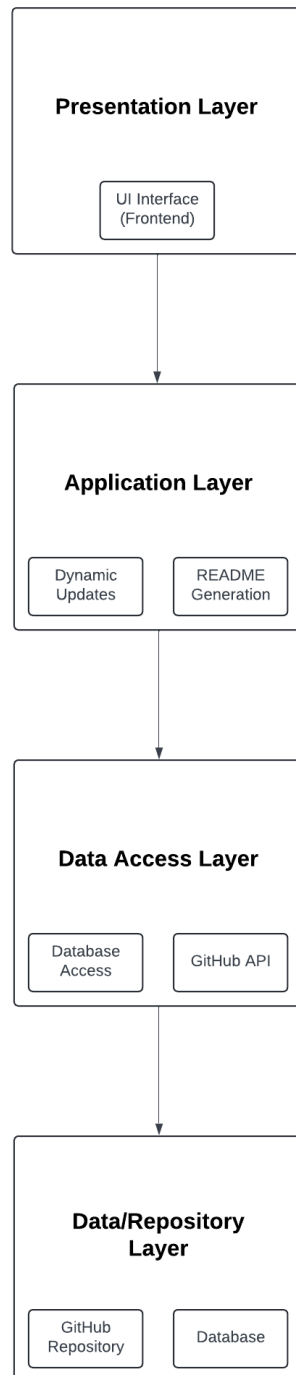


Figure 3.10: Layer Diagram for CodeNarrator

3.3.6 Structure Chart

Below is the Structure Chart showing the system's hierarchical structure:

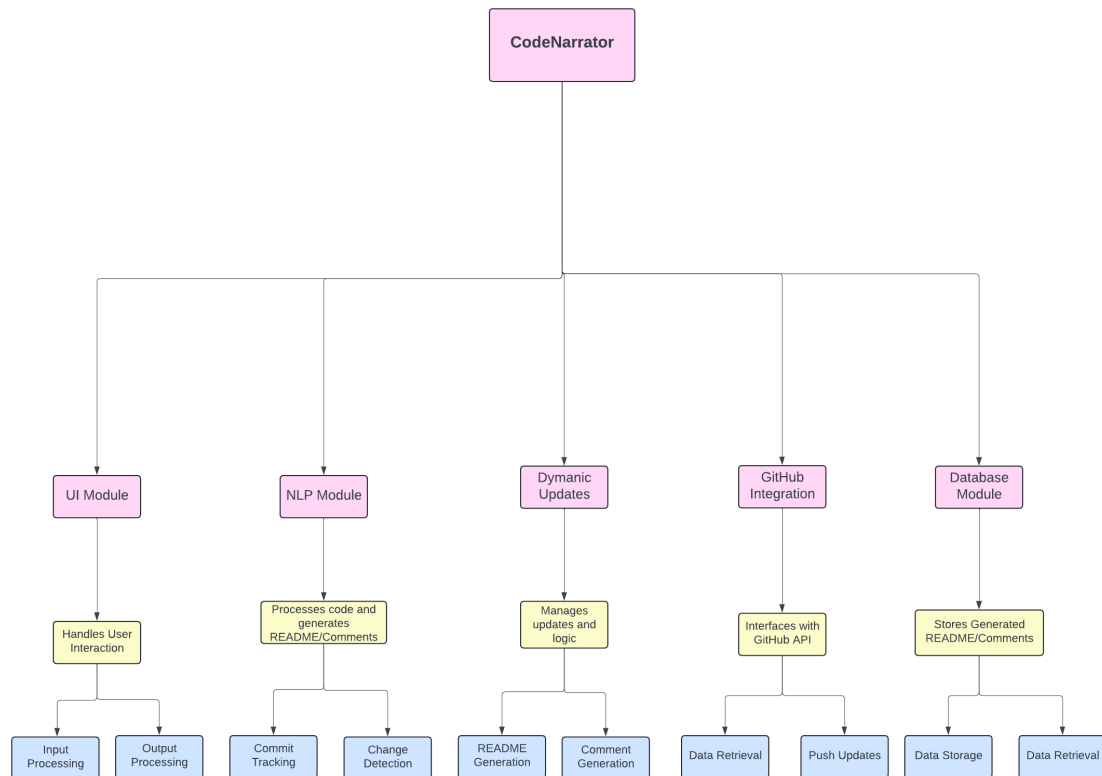


Figure 3.11: Structure Chart for CodeNarrator

3.3.7 Data Representation Diagram

Below is the Data Representation Diagram showing the structure and flow of data within the system:

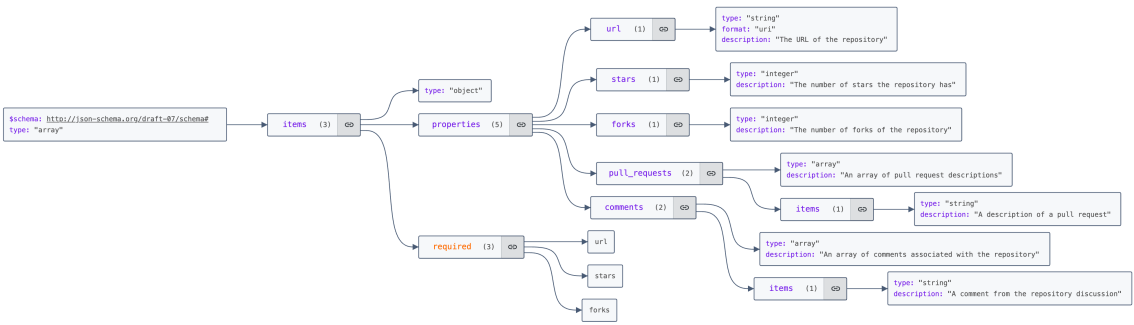


Figure 3.12: Data Representation Diagram for CodeNarrator

3.4 Data Design

3.4.1 Data Dictionary

3.4.1.1 Developer Table

Field Name	Data Type	Description	Constraints
developerID	int	Unique identifier for each developer (Primary Key)	Primary Key, Auto-increment
name	String	Full name of the developer	Not Null
email	String	Email address of the developer	Not Null, Unique
password	String	Password for developer login	Not Null

Table 3.1: Developer Table

3.4.1.2 Repository Table

Field Name	Data Type	Description	Constraints
repositoryID	int	Unique identifier for each repository (Primary Key)	Primary Key, Auto-increment
repositoryURL	string	Link to the repository	Not Null
repoName	String	Name of the repository	Not Null
description	String	Brief description of the repository	Nullable
developerID	int	Reference to the developer who owns the repository (Foreign Key)	Foreign Key (Developer)

Table 3.2: Repository Table

3.4.1.3 Commit Table

Field Name	Data Type	Description	Constraints
commitID	int	Unique identifier for each commit	Primary Key, Auto-increment
commitMessage	String	Message describing the changes in the commit	Not Null
commitDate	Date	Date when the commit was made	Not Null
authorID	int	Reference to the developer who authored the commit	Foreign Key (Developer)
repositoryID	int	Reference to the repository associated with the commit	Foreign Key (Repository)

Table 3.3: Commit Table

3.4.1.4 README Table

Field Name	Data Type	Description	Constraints
readmeID	int	Unique identifier for each README file	Primary Key, Auto-increment
content	String	The content of the README file	Not Null
lastUpdated	Date	The date when the README was last updated	Not Null
repositoryID	int	Reference to the repository to which the README belongs	Foreign Key (Repository)

Table 3.4: README Table

3.4.1.5 CodeComment Table

Field Name	Data Type	Description	Constraints
commentID	int	Unique identifier for each code comment	Primary Key, Auto-increment
lineNumber	int	Line number in the code where the comment is inserted	Not Null
content	String	Content of the comment	Not Null
repositoryID	int	Reference to the repository associated with the comment	Foreign Key (Repository)

Table 3.5: CodeComment Table

3.4.1.6 GitHub Integration Table

Field Name	Data Type	Description	Constraints
integrationID	int	Unique identifier for the GitHub integration	Primary Key, Auto-increment
token	String	GitHub token used for authenticating the integration	Not Null
syncDate	Date	Date when the repository was last synced with GitHub	Not Null
repositoryID	int	Reference to the repository associated with this integration	Foreign Key (Repository)

Table 3.6: GitHub Integration Table

3.4.1.7 Relationships between Tables

- **Developer - Repository:** A developer can own multiple repositories (**1 : M**)
- **Repository - Commit:** A repository can have multiple commits (**1 : M**)
- **Repository - README:** Each repository has one README file (**1 : 1**)
- **Repository - CodeComment:** A repository can have multiple code comments (**1 : M**)
- **Developer - Commit:** A developer can make multiple commits (**1 : M**)

References

SupStat. (2017, July 4). *GitHub Profiler: a tool for repository evaluation*. Data Science Central. Retrieved from: <https://www.datasciencecentral.com/github-profiler-a-tool-for-repository-evaluation>

Hunterkimmnett. (n.d.). *README-Classification-Machine-Learning/README Classification Report.pdf at main · hunterkimmnett/README-Classification-Machine-Learning*. GitHub. Retrieved from: <https://github.com/hunterkimmnett/README-Classification-Machine-Learning/blob/main/README%20Classification%20Report.pdf>

Koreeda, Y., Morishita, T., Imaichi, O., Sogawa, Y. (2023, August). *LARCH: Large Language Model-based Automatic Readme Creation with Heuristics*. ResearchGate. Retrieved from: https://www.researchgate.net/publication/372961688_LARCH_Large_Language_Model_-_based_Automatic_Readme_Creation_with_Heuristics

Wang, T., Wang, S., Chen, T. H. (2023, November). *Study the correlation between the README file of GitHub projects and their popularity*. Journal of Systems and Software, Volume 205, 111806. Retrieved from: <https://www.sciencedirect.com/science/article/abs/pii/S0164121223002017>

Salis, V., Sotiropoulos, T., Louridas, P., Spinellis, D. (2021). *PyCG: Practical Call Graph Generation in Python*. 2021 IEEE/ACM 43rd International Conference on Software Engineering (ICSE). IEEE/ACM. Retrieved from: <https://doi.org/10.1109/ICSE43902.2021.00146>

GitHub. (2023). *GitHub - github/github-app-js-sample: Sample of a GitHub App that comments new pull requests*. GitHub. Retrieved from: <https://github.com/github/github-app-js-sample>

GitHub Docs. (2022). *Building a GitHub App that responds to webhook events - GitHub Docs*. Retrieved from: <https://docs.github.com/en/apps/creating-github-apps/writing->

code-for-a-github-app/building-a-github-app-that-responds-to-webhook-events

GitHub Docs. (2024). *Quickstart for building GitHub Apps* - *GitHub Docs*. Retrieved from: <https://docs.github.com/en/apps/creating-github-apps/writing-code-for-a-github-app/quickstart>

GitHub Docs. (n.d.). *Registering a GitHub App*. Retrieved from: <https://docs.github.com/en/apps/creating-github-apps/registering-a-github-app/registering-a-github-app>